

Quick start

 Copy markdown

Have five minutes? Run this command in your terminal to explore tldraw [starter kits](#):

```
npm create tldraw@latest
```

Have a little more time? Let's try out the tldraw SDK in a React project.

If you're new to React, we recommend using a [Vite template](#) as a starter. We'll assume your project is already running locally.

Getting started

First, install the `tldraw` package from npm:

```
npm install tldraw
```

Next, in your React project, import the `Tldraw` component and tldraw's CSS styles. Then render the `Tldraw` component inside a full screen container:

```
import { Tldraw } from 'tldraw'
import 'tldraw/tldraw.css'

export default function App() {
  return (
    <div style={{ position: 'fixed', inset: 0 }}>
      <Tldraw />
    </div>
  )
}
```

```
)  
}
```

That's pretty much it! At this point, you should have a complete working single-user canvas. You can draw, write, add images and video, zoom, pan, copy and paste, undo and redo—everything you'd expect from a canvas.

You'll be starting from our default [shapes](#), [tools](#), and [user interface](#), but you can customize all of these things for your project if you wish. For now, let's show off a few more features.

Local persistence

Let's add local persistence by passing a `persistenceKey` prop to the `Tldraw` component:

```
export default function App() {  
  return (  
    <div style={{ position: 'fixed', inset: 0 }}>  
      <Tldraw persistenceKey="example" />  
    </div>  
  )  
}
```

Using a `persistenceKey` will use your browser's storage to ensure that your project can survive a browser refresh. It will also synchronize the project between other instances that share the same `persistenceKey` —including in other browser tabs! Give it a try by opening your app in a second window.

Real-time collaboration

To add support for multiple users collaborating in realtime, you can use the [tldraw sync](#) demo library. This library adds real-time collaboration using temporary rooms.

First, install the `@tldraw/sync` package:

```
npm install @tldraw/sync
```

Next, import the `useSyncDemo` hook from the `@tldraw/sync` package. Call it in your component with a unique ID and pass the store that it returns to the `Tldraw` component:

```
import { Tldraw } from 'tldraw'
import { useSyncDemo } from '@tldraw/sync'
import 'tldraw/tldraw.css'

export default function App() {
  const store = useSyncDemo({ roomId: 'insert-any-string-here' })

  return (
    <div style={{ position: 'fixed', inset: 0 }}>
      <Tldraw store={store} />
    </div>
  )
}
```

Try it out by opening your project in a second incognito window, or else access it from another device. You should see all of tldraw's multiplayer features: live cursors, user names, viewport following, cursor chat, and more.

If you want to go further with real-time collaboration, be sure to check out our guide to the [tldraw sync](#) library.

Controlling the canvas

The tldraw [editor](#) has a runtime JavaScript API. Everything that can happen in tldraw can be done programmatically through the [Editor](#) instance.

For simplicity's sake, let's roll back our persistence and sync code. We can then use the `Tldraw` component's `onMount` callback to get access to the `Editor` instance. We'll use the editor to create a new shape on the canvas, select it, then slowly zoom to it.

```
import { Editor, Tldraw, toRichText } from 'tldraw'
import 'tldraw/tldraw.css'

export default function App() {
  const handleMount = (editor: Editor) => {
    editor.createShape({
      type: 'text',
      x: 200,
      y: 200,
      props: {
        richText: toRichText('Hello world!'),
      },
    })

    editor.selectAll()

    editor.zoomToSelection({
      animation: { duration: 5000 },
    })
  }

  return (
    <div style={{ position: 'fixed', inset: 0 }}>
      <Tldraw onMount={handleMount} />
    </div>
  )
}
```

The Editor is tldraw's main interface for controlling the canvas. Be sure to check out the Editor API documentation for more information on what you can do with it, as well as our guide on using the editor.

Next steps

Now that you've seen how the tldraw canvas works, you can:

- Create your own [shapes](#) and [tools](#)
- Customize the [user interface](#)
- Learn more about the [editor](#)
- Explore our [examples](#)

- Build with our [starter kits](#)

The SDK lets you customize shapes, tools, UI, and more. In addition to our long-form docs, we have dozens of examples in our [examples section](#) that cover more of its functionality. You can run these locally with the tldraw [GitHub repository](#).

Join our community

If you build something great, please share it with us in our [#show-and-tell](#) channel on Discord. Good luck!

Next →
[Installation](#)

	Product	Developers	Community	Company
The infinite canvas SDK	Whiteboard	Quick start guide	GitHub	Videos
	Starter kits		Discord	Careers
	Pricing	Starter kits	X/Twitter	License
	FAQ	Examples	LinkedIn	Trademarks
		Releases	Bluesky	CLA
		Docs	Mastodon	Privacy settings

